PYTHON PROGRAMMING — ASSESSMENT 1

Design, Implement and Validate a Payment Calculator for a Village Milk Co-operative

Student Name: SWETHIKA.R

Register Number: 1921260261

Course: CSA0801

Department: IT

Faculty: SHANMATHI

Course Outcome: CO1-AT-1

Total Marks: 50

Abstract

This report analyses and implements a Python payment calculator for the Ksheera Village Milk Producers' Co-operative. The solution uses named constants, input validation, reusable functions, a separate module, decimal arithmetic for monetary values, automated tests, and a seasonal rate-card extension. The report also compares alternative designs using Big-O notation and records local execution-time measurements. Performance figures are environment-dependent and should be reproduced on the student's computer before final submission.

Problem Statement

The co-operative needs to calculate farmer payments from litres, fat percentage, and SNF percentage. A correct calculator must validate entries, apply the base rate and quality adjustments, calculate ten-day totals, handle invalid input, support rate revisions, and remain maintainable as the system grows from 300 to 150,000 farmers.

Assumptions and Formula

Item	Design decision
Standard values	Fat 3.5%; SNF 8.5%
Step size	0.1 percentage point
Step calculation	Nearest whole step; explicit half-up rounding in the Decimal implementation
Payment rounding	Round each entry payment to two decimal places (paise)
Quantity validation	Must be greater than zero
Fat validation	2.0% through 7.0%, inclusive
SNF validation	Assignment gives no valid range; code uses provisional 0.0%–20.0%, to be confirmed
Shift	MORNING or EVENING
Bonus in Task 12	If ten-day litres exceed 500, ₹0.50 is applied to every litre; this interpretation must be confirmed

```
fat_steps = round((fat - 3.5) / 0.1)
snf_steps = round((snf - 8.5) / 0.1)
rate = 30.00 + fat_steps * 0.40 + snf_steps * 0.30
payment = litres * rate
```

TASK 1 — Understand the Problem and Identify the Data

1.1 Inputs and outputs

Field	Data type	Validation / meaning
Farmer ID	str	Non-empty identifier; preserve leading zeros
Litres	Decimal / float input; Decimal for final design	Must be greater than zero
Fat percentage	Decimal	2.0–7.0 inclusive
SNF percentage	Decimal	Provisional 0.0–20.0; confirm business rule
Shift	str	MORNING or EVENING
Rate per litre	Decimal	Calculated from rate card
Payment amount	Decimal	Rounded to two decimal places
Ten-day total	Decimal	Sum of rounded entry payments
Bonus eligibility	bool	True only when ten-day quantity > 500 litres

1.2 Named constants

```
BASE_RATE = Decimal('30.00')
FAT_STEP_VALUE = Decimal('0.40')
SNF_STEP_VALUE = Decimal('0.30')
STANDARD_FAT = Decimal('3.5')
STANDARD_SNF = Decimal('8.5')
```

```
FAT_MIN = Decimal('2.0')
FAT_MAX = Decimal('7.0')
```

1.3 Alternative design and complexity

Separate variables such as litres1, fat1 and snf1 are manageable for one farmer but require new variable names and repeated code for every entry. For n entries, storing named variables manually still requires Θ(n) storage, but the program structure is not scalable and processing logic is awkward. A list of records also uses Θ(n) storage when all entries must be retained, but a loop processes each record in Θ(n) total time. A running total can use Θ(1) additional storage when individual records do not need to be retained. Lists or structured records are therefore more scalable.

Evidence

The field table, named constants, validation rules, and assumptions above constitute the analysis evidence. Add a screenshot of the code or notebook used to demonstrate these values.

TASK 2 — Explore Values and Types in the Python Interpreter

2.1 Types and arithmetic

Expression	Result / type
type(10)	int
type(10.0)	float
type('10')	str
type(True)	bool
type([10, 8.5])	list
7 / 2	3.5
7 // 2	3
7 % 2	1
-7 // 2	-4 (floor division rounds down)
0.1 + 0.2 == 0.3	False

2.2 Completed TODO

```
from decimal import Decimal

payments = ["33.40"] * 1000 + ["0.10"] * 3

def total_float(items):
    total = 0.0
    for amount in items:
        total += float(amount)
    return total

def total_decimal(items):
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
    return total

print("float  :", total_float(payments))
print("Decimal:", total_decimal(payments))
print(0.1 + 0.2 == 0.3,
      Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))

Observed output on the report-generation runtime:
float  : 33400.30000000061
Decimal: 33400.30
False True
```

Binary floating-point cannot represent many decimal fractions exactly, so repeated additions can accumulate tiny errors. Decimal('33.40') constructs the exact decimal value from text. Decimal(float('0.1')) instead captures the float's existing binary

approximation and produces a long decimal expansion; converting to Decimal after converting to float does not restore the original exact decimal value.

2.3 Monetary representations

Approach	Advantages	Limitations
Decimal from strings	Decimal arithmetic follows decimal digits; clear monetary intent	Generally slower and uses more memory than a float
Integer paise	Exact addition for amounts already expressed in paise; fast integer arithmetic	Requires conversions; fractional-paise rules must be explicit
Float + round after each addition	Simple; round() gives a display-friendly result	Binary representation persists; repeated rounding can bias totals

All three approaches take Θ(n) time to total n amounts. The accumulator uses Θ(1) auxiliary space; the list of n input amounts uses Θ(n) space. sys.getsizeof() can measure the shallow size of an object in the current Python runtime, but it does not include all nested objects unless those are measured separately. Do not report a universal byte count: run sys.getsizeof(Decimal('33.40')), sys.getsizeof(3340), and sys.getsizeof(33.4) on the target Python version.

```
import sys
from decimal import Decimal
print(sys.getsizeof(Decimal('33.40')))
print(sys.getsizeof(3340))
print(sys.getsizeof(33.4))
```

TASK 3 — Choose Variables and Name the Constants

```
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
STANDARD_FAT = 3.5
STANDARD_SNF = 8.5
FAT_MIN = 2.0
FAT_MAX = 7.0

value = 10
value = 'ten'  # dynamic typing permits reassignment to another type
a, b = 1, 2
a, b = b, a  # now a=2 and b=1
```

Python does not enforce constants. UPPER_CASE names communicate that values should not be reassigned. Magic numbers make rate changes error-prone because every occurrence must be located. Named constants centralise each value, while a dictionary or configuration object is convenient when rates are selected by category or season. All approaches have O(1) lookup/update for a fixed number of values; the main benefit is maintainability rather than asymptotic speed.

Approach	Rate update	Runtime / memory
Magic numbers	Edit every occurrence; high omission risk	No meaningful speed advantage; constants repeated in source
Named constants	Update one definition per rate	O(1) access; negligible extra memory
RATE_CARD dictionary	Update key/value in one structure; supports multiple cards	Average O(1) key lookup; O(k) storage for k keys

TASK 4 — Write Correct Expressions

```
fat_steps = round((fat - STANDARD_FAT) / 0.1)
snf_steps = round((snf - STANDARD_SNF) / 0.1)
rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE
payment = round(litres * rate, 2)
```

Input	Fat steps	SNF steps	Rate
fat=4.2, SNF=8.7	round((4.2-3.5)/0.1)=7	round((8.7-8.5)/0.1)=2	30 + 7×0.40 + 2×0.30 = ₹33.40
fat=3.1, SNF=8.3	round(-4.0)=-4	round(-2.0)=-2	30 − 4×0.40 − 2×0.30 = ₹27.80

Unexpected result 1: (4.2 - 3.5) / 0.1 may evaluate to 7.000000000000001 due to binary floating-point. Unexpected result 2: 0.1 + 0.2 == 0.3 is False. Unexpected result 3: -7 // 2 is -4 rather than -3 because floor division rounds toward negative infinity.

Operator precedence: exponentiation binds tightly and is right-associative, so 2 ** 3 ** 2 means 2 ** (3 ** 2). Unary minus interacts with exponentiation: -2 ** 2 is -(2 ** 2), or -4, while (-2) ** 2 is 4. Parentheses make intent explicit.

Input fat	Raw step expression	round()	int()	math.floor()
4.2	about 7.000000000000001	7	7	7
3.1	about -4.000000000000000	-4	-4	-4
3.14	about -3.6	-4	-3	-4
4.14	about 6.4	6	6	6

int() truncates toward zero, whereas math.floor() rounds down toward negative infinity. For fat=3.14, the non-integral step is approximately −3.6: int gives −3, while floor and nearest-step rounding give −4. Each operation is O(1) time and O(1) space. Chained comparison 2.0 <= fat <= 7.0 is equivalent to checking both bounds; and/or/not combine Boolean conditions.

TASK 5 — Read Input and Convert Types

```
def read_number(prompt):
    while True:
        raw = input(prompt).strip()
        try:
            return Decimal(raw)
        except InvalidOperation:
            print("Enter a number such as 8.5, not 'eight' or '8,5'.")
```

input() always returns a string because keyboard input is text. Convert identifiers to strings, quantities and percentages to numeric types, and catch ValueError (or Decimal's InvalidOperation when using Decimal). '8,5' is not accepted as a standard decimal string, 'eight' is invalid, and ' 8.5 ' becomes '8.5' after strip(). String pre-validation can reject obvious invalid characters but may reject valid formats or accept malformed combinations unless carefully designed. try/except directly tests whether conversion succeeds. Both methods are O(m) per input string of length m; conversion is typically O(m).

```
from decimal import Decimal, InvalidOperation
raw = input("Litres: ").strip()
try:
    litres = Decimal(raw)
    if litres <= 0:
        raise ValueError("Litres must be greater than zero")
    print(f"Milk quantity: {litres:8.2f} L")
except (InvalidOperation, ValueError) as exc:
    print("Invalid input:", exc)
```

Formatted output such as f'{amount:8.2f}' reserves eight character positions and displays two decimal places. The full interactive example is in main.py.

TASK 6 — Organise the Logic into Functions

```
def rate_per_litre(fat, snf):
    """Calculate and return the price per litre."""
    ...

def payout(litres, fat, snf):
    """Calculate and return the payment for one entry."""
    ...

def ten_day_total(entries):
```

```
    """Return the total payment for a farmer's ten-day entries."""
    ...
```

A function should return a value so the caller can test it, store it, combine it, or print it later. print() displays text but returns None. Parameters and variables assigned inside a function are local by default. Constants can be read inside functions; changing global constants inside functions creates hidden side effects and makes testing harder.

Approach	Assessment	Complexity
One long script	Quick for tiny examples; duplicated logic and difficult tests	O(n) processing, harder maintenance
Reusable functions	Clear responsibilities, reusable and testable	O(n) for n entries; function calls add constant overhead per call
Lambda expressions	Useful for short callbacks/key functions	Not suitable for multi-step validation and documentation

Function-based design does not change the overall asymptotic complexity; it improves readability and correctness.

TASK 7 — Build and Use a Python Module

The completed milkrate.py module is included in the project. It defines the rate constants and functions and uses if __name__ == '__main__' to run the demonstration only when executed directly.

```
import milkrate
from milkrate import payout
import milkrate as mr

print(milkrate.payout(8.5, 4.2, 8.7))
print(payout(8.5, 4.2, 8.7))
print(mr.payout(8.5, 4.2, 8.7))

Expected demonstration output:
   10 L  fat 3.5  snf 8.5  rate  30.00  pay   300.00
  8.5 L  fat 4.2  snf 8.7  rate  33.40  pay   283.90
   12 L  fat 3.1  snf 8.3  rate  27.80  pay   333.60
```

When imported, the module's __name__ is 'milkrate', so the guarded demonstration does not execute. Python searches paths listed in sys.path, including the script's directory and standard installation paths. Imported modules are cached in sys.modules so repeated imports usually reuse the loaded module rather than executing its top-level code again. Importing a shared module makes rate revisions safer than maintaining copied versions in multiple scripts. The first import has a one-time loading cost; subsequent imports in the same process are typically lookups.

math supplies mathematical functions; decimal supplies decimal arithmetic suitable for monetary calculations.

TASK 8 — Plan the Algorithm Before Coding

8.1 Pseudocode

```
BEGIN
  READ farmer_id, shift, litres, fat, snf
  IF litres <= 0 THEN report invalid quantity and STOP
  IF fat < 2.0 OR fat > 7.0 THEN report invalid fat and STOP
  IF SNF outside configured limits THEN report invalid SNF and STOP
  fat_steps <- nearest_integer((fat - 3.5) / 0.1)
  snf_steps <- nearest_integer((snf - 8.5) / 0.1)
  rate <- 30.00 + fat_steps*0.40 + snf_steps*0.30
  payment <- round_to_paise(litres * rate)
  DISPLAY farmer_id, rate, payment
END
```

8.2 Flowchart

```
[Start]
   |
[Read entry]
   |
{Input valid?} --No--> [Display error] --> [End]
   |
  Yes
   |
[Calculate fat/SNF steps]
   |
[Calculate rate and payment]
   |
[Display result]
   |
 [End]
```

8.3 Hand trace for five entries

Litres	Fat	SNF	Rate (₹/L)	Payment (₹)
10	3.5	8.5	30.00	300.00
8.5	4.2	8.7	33.40	283.90
12	3.1	8.3	27.80	333.60
5	2.0	8.5	24.00	120.00
20	7.0	8.5	44.00	880.00

Total of the five rounded payments: ₹1917.50.

TASK 9 — Implement and Test the Calculator

The main implementation is split between milkrate.py (business logic) and main.py (interactive input/output). Decimal is used for money; input validation is centralised in validate_entry().

9.1 Test table

Test case	Expected	Actual	Status
Standard values: 10 L, 3.5%, 8.5%	₹300.00	₹300.00	PASS
Above standard: 8.5 L, 4.2%, 8.7%	₹283.90	₹283.90	PASS
Below standard: 12 L, 3.1%, 8.3%	₹333.60	₹333.60	PASS
Lowest fat: 10 L, 2.0%, 8.5%	₹240.00	₹240.00	PASS
Highest fat: 10 L, 7.0%, 8.5%	₹440.00	₹440.00	PASS
Invalid text 'eight'	Reject input	Rejected by Decimal conversion	PASS
Comma decimal '8,5'	Reject input	Rejected by Decimal conversion	PASS
Negative quantity -1 L	Reject input	ValueError	PASS
Fat 7.1%	Reject input	ValueError	PASS
Fat 2.0%, 10 L	₹240.00	₹240.00	PASS

9.2 Five-entry total

Entry	Input (L, fat, SNF)	Rate (₹/L)	Payment (₹)
1	10, 3.5, 8.5	30.00	300.00
2	8.5, 4.2, 8.7	33.40	283.90
3	12, 3.1, 8.3	27.80	333.60
4	5, 2.0, 8.5	24.00	120.00
5	20, 7.0, 8.5	44.00	880.00
Total			1917.50

Manual testing is useful for exploratory cases but becomes repetitive after changes. Automated tests use assert statements to rerun known cases quickly and consistently. A test suite with m cases takes O(m) test executions; each simple payment assertion is O(1).

TASK 10 — Compare Expectations with Execution

The following measurements were collected with time.perf_counter() in the report-generation runtime. They are examples of measured runs, not guaranteed timings on another computer. Rerun the included experiment on the submission machine and replace the times if they differ.

Entries	Float total (₹)	Float time (s)	Decimal total (₹)	Decimal time (s)
300	10020.00	0.000034	10020.00	0.000102
6000	200400.00	0.000412	200400.00	0.001226
150000	5010000.00	0.010169	5010000.00	0.038075

Task 2's 1,003-item test produced float 33400.30000000061 and Decimal 33400.30. The float error is tiny but demonstrates that the binary result is not an exact decimal monetary total. Decimal is exact for these decimal inputs and operations, subject to the chosen rounding rules.

For a bank payment, the policy should specify whether each farmer's payment is rounded to paise before aggregation. This report rounds each individual payment to two decimal places and then sums those amounts, because the amount payable to each farmer is a bank-facing monetary value. Rounding only the final sum can differ by a few paise when many fractional-paise values are present.

Rate cache alternative

A dictionary keyed by (fat, snf) can cache a rate after its first calculation. For k distinct pairs among n entries, building the cache takes O(k) calculations and subsequent lookups are average O(1), giving roughly O(n+k) processing. Recalculating each rate costs O(n) rate calculations. The cache uses O(k) extra memory. It is worthwhile only when repeated pairs are common and rounding/normalisation creates stable keys.

TASK 11 — Recommend and Defend the Final Design

- Use Decimal created from text for monetary values and round each entry payment to two decimal places with a documented policy.
- Use named constants or a rate-card configuration rather than magic numbers.
- Keep input/output separate from business logic and use functions in milkrate.py.
- Validate quantity, fat, SNF, and shift before calculating a payment.
- Retain transaction records in a database or durable file for auditability; a running total alone is insufficient for disputes and payment reconciliation.
- The arithmetic work is O(n) for n entries. 150,000 entries are computationally practical on an ordinary computer, but production suitability also depends on database I/O, backups, concurrency, access controls, audit trails, and recovery procedures.

A text or JSON rate card can be loaded at startup. Loading t rate values takes O(t) time and O(t) memory; editing rates becomes easier and does not require editing the calculation logic. The file must be validated, versioned, backed up, and protected from unauthorised changes. For the small number of rates here, the startup cost is negligible.

Limitations: SNF limits were not specified, so the included 0–20% limits are provisional; the production system must confirm them. The assignment does not specify whether fat/SNF are rounded or truncated by the analyser, nor whether the bonus applies to all litres or only litres above 500. Those rules require approval before deployment.

TASK 12 — DAY-4 DESIGN CHALLENGE: Re-evaluate and Adapt the Design

A. Impact Analysis

April, May, and June use a ₹32.00 base rate and ₹0.45 per fat step; the SNF step remains ₹0.30 because no seasonal SNF change is specified. The analyser now supplies readings to two decimal places. The step size remains 0.1 percentage point, so each reading is converted to the nearest whole 0.1% step. The bonus condition is strictly greater than 500 litres over ten days.

B. Worked examples

Case	Calculation	Result
April, fat 4.20%, SNF 8.70%	Fat steps 7; SNF steps 2; 32 + 7×0.45 + 2×0.30	₹35.75/L
April, 8.50 L at that quality	8.50 × 35.75	₹303.88
January, same quality	30 + 7×0.40 + 2×0.30	₹33.40/L
Ten-day total 500 L in April	500 is not greater than 500; no bonus	Bonus ₹0.00
Ten-day total 510 L in April	510 × ₹0.50 bonus	Bonus ₹255.00

For 510 litres all at ₹35.75 per litre, milk payment is ₹18,232.50 and bonus is ₹255.00, giving ₹18,487.50. The code assumes the bonus applies to every litre once the threshold is crossed; confirm that interpretation with the co-operative.

C. Adaptation

The original module remains appropriate but is extended with seasonal_rate_per_litre(), seasonal_payout(), and ten_day_total_with_bonus(). Season is passed as a month number. The rate logic remains centralised and testable; only the rate card and bonus rule change.

```
from milkrate import seasonal_rate_per_litre, seasonal_payout, ten_day_total_with_bonus

print(seasonal_rate_per_litre("4.2", "8.7", 4))  # 35.75
print(seasonal_payout("8.5", "4.2", "8.7", 4))  # 303.88

entries = [(170, "4.2", "8.7"),
           (170, "4.2", "8.7"),
           (170, "4.2", "8.7")]
print(ten_day_total_with_bonus(entries, 4))
# (510.00, 18232.50, 255.00, 18487.50)
```

D. Revised test table

Test	Expected result	Design outcome
January, 8.5 L, 4.2/8.7	₹283.90	Original rates retained
April, 8.5 L, 4.2/8.7	₹303.88	Seasonal rates selected
April, exactly 500 L	No bonus	Strict >500 condition enforced
April, 510 L at 4.2/8.7	₹18,232.50 milk + ₹255.00 bonus = ₹18,487.50	Bonus applied to all litres above threshold

Decision: modify rather than completely redesign. The existing module, functions, validation, and Decimal money handling are reusable. A separate rate-selection function and bonus function make seasonal changes explicit and reduce duplicated logic. The main trade-off is extra configuration and tests; the core per-entry complexity remains O(1), and a ten-day total remains O(n).

Conclusion

The assignment demonstrates how structured problem analysis, meaningful constants, careful numeric types, input validation, reusable functions, modules, pseudocode, testing, and complexity analysis produce a reliable payment calculator. Decimal arithmetic avoids binary floating-point errors in monetary totals, while a modular design makes rate revisions safer. The extended design supports seasonal rates and a ten-day volume bonus. Before real deployment, the co-operative must approve SNF validation bounds, analyser rounding policy, bonus interpretation, and operational controls.

README and Submission Checklist

- Replace placeholders for course, department, and faculty details.
- Run python milkrate.py and python use_milkrate.py.
- Run python Task09/test_milkrate.py from the project root.
- Run python Task12/seasonal_demo.py from the project root.
- Capture screenshots from your own terminal or IDE; do not present generated text as screenshots.
- Rerun timing experiments on your own computer and update the table if necessary.

- Create one progressive Git commit per task.
- Push the repository and verify that faculty can access it.

Appendix A — Complete milkrate.py

```
"""Payment calculations for the Ksheera Village Milk Producers' Co-operative."""
from decimal import Decimal, ROUND_HALF_UP

BASE_RATE = Decimal("30.00")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
STANDARD_FAT = Decimal("3.5")
STANDARD_SNF = Decimal("8.5")
FAT_MIN = Decimal("2.0")
FAT_MAX = Decimal("7.0")
SNF_MIN = Decimal("0.0")       # Assumption: no narrower SNF limits were supplied.
SNF_MAX = Decimal("20.0")     # Operational bound; confirm with the co-operative.
CENT = Decimal("0.01")
STEP = Decimal("0.1")

def money(value):
    """Round a monetary value to two decimal places using half-up rounding."""
    return Decimal(value).quantize(CENT, rounding=ROUND_HALF_UP)

def validate_entry(litres, fat, snf, shift=None):
    """Raise ValueError when quantity, quality readings, or shift are invalid."""
    litres, fat, snf = Decimal(str(litres)), Decimal(str(fat)), Decimal(str(snf))
    if litres <= 0:
        raise ValueError("Milk quantity must be greater than zero.")
    if not (FAT_MIN <= fat <= FAT_MAX):
        raise ValueError(f"Fat must be between {FAT_MIN}% and {FAT_MAX}%.")
    if not (SNF_MIN <= snf <= SNF_MAX):
        raise ValueError(f"SNF must be between {SNF_MIN}% and {SNF_MAX}%.")
    if shift is not None and shift.upper() not in {"MORNING", "EVENING"}:
        raise ValueError("Shift must be MORNING or EVENING.")
    return litres, fat, snf

def rate_per_litre(fat, snf, *, base_rate=BASE_RATE,
                   fat_step_value=FAT_STEP_VALUE,
                   snf_step_value=SNF_STEP_VALUE):
    """Return the rate per litre for fat/SNF readings using nearest 0.1% steps."""
    fat, snf = Decimal(str(fat)), Decimal(str(snf))
    fat_steps = int(((fat - STANDARD_FAT) / STEP).quantize(Decimal("1"), rounding=ROUND_HALF_UP))
    snf_steps = int(((snf - STANDARD_SNF) / STEP).quantize(Decimal("1"), rounding=ROUND_HALF_UP))
    rate = Decimal(str(base_rate)) + fat_steps * Decimal(str(fat_step_value)) + snf_steps *
Decimal(str(snf_step_value))
    return money(rate)

def payout(litres, fat, snf, *, base_rate=BASE_RATE,
           fat_step_value=FAT_STEP_VALUE, snf_step_value=SNF_STEP_VALUE):
    """Return one entry's payment rounded to paise."""
    litres = Decimal(str(litres))
    rate = rate_per_litre(fat, snf, base_rate=base_rate,
                          fat_step_value=fat_step_value,
                          snf_step_value=snf_step_value)
    return money(litres * rate)

def ten_day_total(entries):
    """Return total payment and litres for a sequence of (litres, fat, snf) entries."""
    total = Decimal("0.00")
    litres_total = Decimal("0.00")
    for litres, fat, snf in entries:
        validate_entry(litres, fat, snf)
```

```
        total += payout(litres, fat, snf)
        litres_total += Decimal(str(litres))
    return money(total), litres_total

def seasonal_rate_per_litre(fat, snf, month):
    """Return rate for a month number; April-June uses seasonal rates."""
    if month in (4, 5, 6):
        return rate_per_litre(fat, snf, base_rate=Decimal("32.00"),
                              fat_step_value=Decimal("0.45"),
                              snf_step_value=SNF_STEP_VALUE)
    return rate_per_litre(fat, snf)

def seasonal_payout(litres, fat, snf, month):
    """Return payment using the selected monthly rate card."""
    litres = Decimal(str(litres))
    return money(litres * seasonal_rate_per_litre(fat, snf, month))

def ten_day_total_with_bonus(entries, month):
    """Return (milk litres, milk payment, bonus, total) for a ten-day period.

    Assumption: if total litres exceed 500, the bonus is paid on every litre
    supplied in that ten-day period, not just litres above 500.
    """
    total_litres = Decimal("0.00")
    milk_payment = Decimal("0.00")
    for litres, fat, snf in entries:
        validate_entry(litres, fat, snf)
        litres_d = Decimal(str(litres))
        total_litres += litres_d
        milk_payment += seasonal_payout(litres_d, fat, snf, month)
    bonus = money(total_litres * Decimal("0.50")) if total_litres > Decimal("500") else Decimal("0.00")
    return total_litres, money(milk_payment), bonus, money(milk_payment + bonus)

if __name__ == "__main__":
    examples = [(10, 3.5, 8.5), (8.5, 4.2, 8.7), (12, 3.1, 8.3)]
    for litres, fat, snf in examples:
        rate = rate_per_litre(fat, snf)
        print(f"{litres:5} L  fat {fat}  snf {snf}  rate {rate:6.2f}  pay {payout(litres, fat, snf):8.2f}")
```

Appendix B — Interactive main.py

```
"""Interactive entry program for the milk payment calculator."""
from decimal import Decimal
from milkrate import validate_entry, payout, rate_per_litre, ten_day_total

def read_decimal(prompt):
    """Read a decimal number, allowing surrounding spaces but not comma decimals."""
    while True:
        raw = input(prompt).strip()
        try:
            return Decimal(raw)
        except Exception:
            print("Invalid number. Enter a number using a dot as decimal separator (e.g. 8.5).")

def main():
    farmer_id = input("Farmer ID: ").strip()
    shift = input("Shift (MORNING/EVENING): ").strip().upper()
    litres = read_decimal("Milk quantity (litres): ")
    fat = read_decimal("Fat percentage: ")
    snf = read_decimal("SNF percentage: ")
    try:
        validate_entry(litres, fat, snf, shift)
```

```
        except ValueError as exc:
            print("Invalid entry:", exc)
            return
        rate = rate_per_litre(fat, snf)
        amount = payout(litres, fat, snf)
        print(f"Farmer: {farmer_id} | Shift: {shift}")
        print(f"Rate per litre: ₹{rate:7.2f}")
        print(f"Payment amount: ₹{amount:9.2f}")

if __name__ == "__main__":
    main()
```

Appendix C — Automated test command

```
python Task09/test_milkrate.py
```